

Object-Oriented Programming — สร้างระบบที่ ขยายได้

บทที่ 18–21: Classes · Inheritance · Design Patterns · OOP Mini-Project

จบ Part นี้ → เปลี่ยน script ที่รันครั้งเดียวเป็น trading system ที่ใช้ได้จริง

🚫 อ่าน Part นี้ยังไ้

● เขียนฟังก์ชันเป็นแล้ว แต่ไม่เคยเขียน class — บทที่ 18.1 เล่าเหตุผลก่อนสอนไวยากรณ์ให้อ่านตรงนั้นให้เข้าใจว่า "ทำไมโค้ดแบบ Part II เริ่มไม่พอ" · แล้วยอมรับไปก่อนว่า self ดูแปลก — พอเขียนไป 2-3 คลาสจะชินเอง · เจอบล็อกโค้ดยาว ๆ ให้ดูตารางหน้าที่ของแต่ละเมธอดก่อนอ่านรายละเอียด

● เขียน OOP ภาษาอื่นมาแล้ว — ไวยากรณ์คุ้นอยู่แล้ว · ของสำหรับคุณคือ ● [รอบสอง] เรื่องกับดักที่ ABC จับไม่ได้ (บังคับได้แค่ "ต้องมีเมธอดชื่อนี้" ไม่รู้ว่าข้างในทำอะไร) และกล่อกับดัก cache กับ @property ที่ทำให้ค่าค้างโดยไม่ี error · สองเรื่องนี้คือบักที่เคยอยู่ในโค้ดของเล่มนี้เองจริง ๆ

ทำไมต้องเรียน OOP?

โค้ดจาก Part II ทำงานได้ดีกับ pair เดียว แต่พอเพิ่มกลยุทธ์หรือสินทรัพย์ใหม่จะเริ่มยุ่งยาก: ต้องแก้โค้ดหลายที่พร้อมกัน, copy-paste แล้วแก้ทีละจุด, และตัวแปรกระจายไปทั่วโปรเจกต์

OOP แก้ปัญหาเหล่านี้ด้วยการ จัดกลุ่มข้อมูลและ behavior เข้าด้วยกันเป็น class — เปลี่ยนทีเดียวกระทบทุกที่

Running Example ตลอด Part III — Refactor PairTradingSignal จาก Part II

เราจะนำ PairTradingSignal จาก Part II มาออกแบบใหม่ให้เป็น OOP ที่ดี:

บท 18 → เข้าใจ class พื้นฐาน · บท 19 → Abstract base class + inheritance

บท 20 → ใส่ design patterns · บท 21 → Mini-project: ระบบครบวงจร

บทที่ 18: Classes & Objects

แก่นของบทนี้

Class คือ "blueprint" — นิยามว่า object ประเภทนี้เก็บข้อมูลอะไร (attributes) และทำอะไรได้บ้าง (methods) Object คือ "ชิ้นงานจริง" ที่สร้างจาก blueprint นั้น เหมือนแบบบ้าน (class) กับบ้านจริง (object)

18.1 ทำไมต้องมี Class — เรื่องเล่าจาก Pair ที่สอง

โค้ดสไตล์ Part I/II (ตัวแปร + ฟังก์ชัน) ทำงานได้ดีกับ pair เดียว:

```
# Part II style: ตัวแปรวางไว้ระดับบนสุดของไฟล์ window = 60 entry_z = 2.0 df =  
build_spread(btc_bybit, btc_binance, window) hl = calc_half-life(df["spread"])  
# สมมติว่า make_signal คือฟังก์ชันสร้างสัญญาณที่เราเขียนไว้แล้วใน Part II signal =  
make_signal(df, entry_z)
```

ปัญหาเริ่มเมื่อเทรด **pair ที่สอง** ซึ่งใช้ parameter คนละชุด:

```
# เพิ่ม pair ที่สอง... ตัวแปรเริ่มแตกหน่อ window_2 = 90 # pair นี้ mean-revert ชั่วๆ ใช้ window  
ยาวกว่า entry_z_2 = 1.5 df_2 = build_spread(eth_okx, eth_bybit, window_2) hl_2 =  
calc_half-life(df_2["spread"]) signal = make_signal(df, entry_z) signal_2 =  
make_signal(df_2, entry_z) # ← bug! ลืมเปลี่ยนเป็น entry_z_2 # Python ไม่ฟ้องอะไรเลย — signal  
ผิดพลาด
```

สังเกตว่า bug นี้เกิดเพราะข้อมูลกับ parameter ของแต่ละ pair ไม่ได้ผูกกัน — มันแค่ "วางอยู่ใกล้กัน" ในไฟล์ แล้วเราต้องจำเองว่าอะไรคู่กับอะไร ลองนึกต่อ: ถ้ามี 10 pairs จะมีตัวแปร 40+ ตัวลอยอยู่ และทุกการเรียกฟังก์ชันคือโอกาสหยิบผิดคู่

Class คือคำตอบ: กล่องที่มัด "ข้อมูล + parameter + ฟังก์ชันที่ทำงานกับมัน" ไว้ด้วยกันเป็นชิ้นเดียว แล้วสร้างก็กล่องก็ได้:

```
# แต่ละ pair คือ object หนึ่งชิ้น — ถือของของตัวเองครบ pair1 = PairStrategy("BTC-Bybit", "BTC-  
Binance", window=60, entry_z=2.0) pair2 = PairStrategy("ETH-OKX", "ETH-Bybit",  
window=90, entry_z=1.5) signal1 = pair1.make_signal() # ใช้ window=60, entry_z=2.0 ของ  
ตัวเองเสมอ signal2 = pair2.make_signal() # ใช้ของตัวเอง — หยิบสลับกันไม่ได้โดยโครงสร้าง
```

📌 **อ่านว่า:** สั่งให้ make_signal ของ pair1 ทำงาน — จุด (.) ยังอ่านว่า "ของ" เหมือนเดิมจาก Part I แค่นี้ "ของ" หมายถึง

CLASS ไม่ใช่ความรู้ใหม่

Class = dict (เก็บข้อมูล) + function (ทำงาน) เอามาผูกติดกัน — สองอย่างที่ใช้คล่องแล้วจาก Part I

ความใหม่มีแค่ 2 สิ่ง:

1. `self = "กล่องใบนี้"` — เมื่อเราพิมพ์ `pair1.make_signal()` Python จะแปลงเป็น `PairStrategy.make_signal(pair1)` ให้อัตโนมัติ

พูดง่าย ๆ คือ `self` ก็คือ `pair1` ที่ถูกส่งเข้ามาเป็น argument แรกเสมอ — เราไม่ต้องส่งเอง Python จัดการให้

2. `self.window = "หยิบ window จาก dict ของกล่องใบนี้"` — ภายในเป็นแค่ dict ธรรมดา แค่ syntax ต่างกัน: `cfg["window"]` กับ `self.window` คือสิ่งเดียวกัน

18.2 Class พื้นฐาน

```
# แบบ procedural (Part I/II style) — ข้อมูลกระจาย symbol_a = "BTCUSDT-Bybit" symbol_b = "BTCUSDT-Binance" window = 60 entry_z = 2.0 # แบบ OOP — ข้อมูลอยู่ในที่เดียว class PairConfig: """เก็บ configuration ของ pair trading""" def __init__(self, symbol_a, symbol_b, window=60, entry_z=2.0, exit_z=0.5): self.symbol_a = symbol_a self.symbol_b = symbol_b self.window = window self.entry_z = entry_z self.exit_z = exit_z def __repr__(self): return (f"PairConfig({self.symbol_a} / {self.symbol_b}, " f"window={self.window}, entry_z=±{self.entry_z})") def is_valid(self): """ตรวจสอบว่า config สมเหตุสมผล""" return (self.window >= 20 and self.entry_z > self.exit_z > 0) # สร้าง object cfg = PairConfig("BTCUSDT-Bybit", "BTCUSDT-Binance", window=60) print(cfg) print(f"Valid: {cfg.is_valid()}")
```

► OUTPUT

```
PairConfig(BTCUSDT-Bybit / BTCUSDT-Binance, window=60, entry_z=±2.0)
Valid: True
```

18.3 ส่วนประกอบของ Class

ส่วนประกอบ	ใช้งาน	ตัวอย่าง
<code>__init__</code>	Constructor — รันเมื่อ <code>obj = Class(...)</code>	เก็บ parameters, สร้าง initial state
<code>__repr__</code>	String representation สำหรับ debug	<code>print(obj)</code> แสดงข้อมูลสำคัญ
<code>self</code>	ชี้ไปยัง object ปัจจุบัน	<code>self.window</code> คือ attribute ของ object นี้
Instance method	Function ที่รับ <code>self</code> เป็น argument แรก	<code>def is_valid(self):</code>
Class method	Method ที่แชร์ระหว่างทุก object	<code>@classmethod def from_dict(cls, d):</code>
Static method	Function ใน class ที่ไม่ใช่ <code>self</code>	<code>@staticmethod def annualize(daily):</code>

self กับ cls ต่างกันอย่างไร?

	self	cls
ชี้ไปที่	object ตัวนั้น (กล่องใบนี้)	class เอง (แบบพิมพ์เขียว)
ใช้ใน	method ทั่วไป	<code>@classmethod</code>
ตัวอย่าง	<code>self.window</code> = window ของ object นี้	<code>cls(rets, name)</code> = สร้าง object ใหม่จาก class

`@classmethod` ใช้บ่อยสำหรับ "alternative constructor" เช่น

`TradingMetrics.from_prices(prices)` = "สร้าง TradingMetrics จาก prices แทนที่จะส่ง returns โดยตรง"

```
# โค้ดตลอด Part III ใช้สองตัวนี้ — import ไว้ครั้งเดียวใช้ได้ทั้ง Part
import numpy as np
import pandas as pd

class TradingMetrics:
    """คำนวณและเก็บ performance metrics"""
    TRADING_DAYS = 252
    # class variable — แชร์ทุก instance
    def __init__(self, returns, name="Strategy"):
        self.returns = np.array(returns)
        self.name = name
        self._cache = {}
        # _ prefix = "ควรใช้ภายใน class" (convention)
    @property
    def sharpe(self):
        """คำนวณ Sharpe Ratio (cached)"""
        if "sharpe" not in self._cache:
            m = self.returns.mean()
            s = self.returns.std()
            self._cache["sharpe"] = m / s * np.sqrt(self.TRADING_DAYS)
        return self._cache["sharpe"]
    @property
    def annual_return(self):
        return self.returns.mean() * self.TRADING_DAYS
    @property
    def annual_vol(self):
        return self.returns.std() * np.sqrt(self.TRADING_DAYS)
    @property
    def max_drawdown(self):
        equity = np.cumprod(1 + self.returns)
        peak = np.maximum.accumulate(equity)
        return (equity - peak) / peak).min()
    @classmethod
    def from_prices(cls, prices, name="Strategy"):
        """สร้าง TradingMetrics จาก price series แทน returns"""
        rets = np.diff(prices) / prices[:-1]
        return cls(rets, name)
    @staticmethod
    def annualize(daily_val, periods=252):
        """Utility: annualize daily metric"""
        return daily_val * np.sqrt(periods)
    def __repr__(self):
        return (f"TradingMetrics({self.name}: " f"Sharpe={self.sharpe:.2f}, "
```

```
f"Return={self.annual_return:.1%}, " f"DD={self.max_drawdown:.1%}")) # ใช้งาน import
numpy as np np.random.seed(42) rets = np.random.normal(0.0005, 0.012, 252) m =
TradingMetrics(rets, "BTC Pair") print(m) print(f"Annual Vol: {m.annual_vol:.1%}")
```

► OUTPUT

```
TradingMetrics(BTC Pair: Sharpe=0.62, Return=11.5%, DD=-16.4%)
Annual Vol: 18.4%
```

🤖 3 เครื่องหมาย 🏠 ที่โผล่พร้อมกัน — ต่างกันที่ "ใครถูกส่งเข้าไปเป็นตัวแรก"

```
@property                                # ① ได้ self
def sharpe(self): ...

@classmethod                             # ② ได้ cls (ตัวคลาสเอง)
def from_prices(cls, prices): ...

@staticmethod                             # ③ ไม่ได้อะไรเลย
def annualize(daily_val): ...
```

ทั้งสามตัวคือ **decorator** — ป้ายที่แปะบนฟังก์ชันเพื่อเปลี่ยนวิธีเรียกใช้ · จำง่ายที่สุดโดยดูว่า **parameter** ตัวแรกคืออะไร:

1. `@property + self` — "อ่านได้เหมือนข้อมูล แต่จริง ๆ คำนวณสด" · เรียก `m.sharpe` ไม่ใส่วงเล็บ · ใช้เมื่อค่า นั้น "เป็นคุณสมบัติของชิ้นงานนี้" เช่น Sharpe ของพอร์ตนี้
2. `@classmethod + cls` — "ผลิตชิ้นงานใหม่ด้วยวิธีอื่น" · `cls` คือตัวคลาส จึงเขียน `return cls(...)` เพื่อสร้าง object ได้ · ที่นี้ `from_prices()` รับราคาแล้วแปลงเป็น `return` ให้ก่อน — เรียกว่า **alternative constructor**: `TradingMetrics.from_prices(p)` ใช้ได้โดยไม่ต้องมี object ก่อน
3. `@staticmethod` — "ฟังก์ชันช่วยที่บังเอิญเก็บไว้ในคลาสนี้" · ไม่แตะ `self` ไม่แตะ `cls` เลย · `annualize()` แปลงค่ารายวันเป็นรายปี — ใครส่งเลขอะไรมาก็คำนวณให้ไม่เกี่ยวกับพอร์ตไหน · เก็บไว้ในคลาส เพราะอยู่เป็นหมวดเดียวกันเท่านั้น

เทียบให้เห็นภาพ: `@property = "คุณสมบัติของชิ้นงานนี้"` · `@classmethod = "โรงงานผลิตชิ้นงาน"` · `@staticmethod = "เครื่องมือที่วางไว้ในลิ้นชักเดียวกัน"`

🔴 **อ่านว่า:** อีกจุดที่ต้องสังเกตใน `sharpe`: `if "sharpe" not in self._cache:` — คำนวณครั้งแรกครั้งเดียว แล้วเก็บคำตอบไว้ เรียกซ้ำก็หยิบของเก่ามาให้ · เรียกว่า **caching** ช่วยเมื่อการคำนวณหนักและถูกเรียกหลายรอบ · ⚠️ แต่มันมีราคา — ดูก่อนถัดไป

⚠️ **cache กับ @property** = คู่ที่ขัดกันเอง ถ้าข้อมูลเปลี่ยนได้

คอมเมนต์ใต้โค้ดบอกว่า "ถ้า `returns` เปลี่ยน → ผลลัพธ์เปลี่ยนอัตโนมัติ" — จริงสำหรับ `annual_vol` (ไม่ cache) แต่ **ไม่จริงสำหรับ sharpe** เพราะมัน cache ไว้แล้ว

```
m = TradingMetrics(rets) print(f"sharpe รอบแรก : {m.sharpe:.4f}") # คำนวณ แล้วเก็บ
ลง cache print(f"vol รอบแรก : {m.annual_vol:.4f}") # เปลี่ยนข้อมูลเป็นชุดที่ผันผวนกว่าเดิม
มาก np.random.seed(7) m.returns = np.random.normal(0.002, 0.030, 252)
print(f"sharpe รอบสอง : {m.sharpe:.4f} <-- ❌ ค่าเดิม! หยิบจาก cache") print(f"vol
รอบสอง : {m.annual_vol:.4f} <-- ✓ ค่าใหม่ เพราะไม่ได้ cache")
```

► OUTPUT

```
sharpe รอบแรก : 0.6233
vol รอบแรก : 0.1839
sharpe รอบสอง : 0.6233 <-- ❌ ค่าเดิม! หยิบจาก cache
vol รอบสอง : 0.4552 <-- ✓ ค่าใหม่ เพราะไม่ได้ cache
```

🚫กฎ: cache ได้เมื่อข้อมูลต้นทางไม่เปลี่ยนหลังสร้าง object (immutable) · ถ้าเปลี่ยนได้ ต้องล้าง cache ตอนเปลี่ยน — วิธีมาตรฐานคือใช้ @property คู่กับ setter ที่สั่ง self._cache.clear() · บั๊กจากการลืมล้าง cache หายากมากเพราะโค้ดไม่ error แค่ให้คำตอบเก่า

18.4 @property — Attribute ที่คำนวณแบบ Lazy

@property ทำให้ method เรียกใช้ได้เหมือน attribute — ไม่ต้องใส่วงเล็บ และคำนวณเมื่อถูกเรียกเท่านั้น

```
# — เทียบให้ดู (สองแบบนี้อยู่ด้วยกันไม่ได้ — เลือกได้แบบเดียว) — # # ถ้า annual_vol เป็น method
ธรรมดา: # print(m.annual_vol()) — ต้องใส่ () ทั้งที่มันคือ "ค่า" ไม่ใช่ "คำสั่ง" # # ถ้าใส่ @property
ทับไว้: print(f"{m.annual_vol:.4f}") # ← อ่านเหมือน attribute · ซัดกว่า # ที่คลาสนี้เลือกแบบหลัง
(ดูบรรทัด @property ในโค้ดข้างบน) # ลองเติม () จะได้ TypeError: 'numpy.float64' object is
not callable # — เพราะ m.annual_vol กลายเป็น "ตัวเลข" ไปแล้ว ไม่ใช่ฟังก์ชัน
```

► OUTPUT

```
0.4552
```

อ่าน error นั้นให้ออก — มันบอกความจริงของ @property ทั้งหมด

TypeError: 'numpy.float64' object is not callable แปลตรงตัวว่า "ของชิ้นนี้เป็นเลขทศนิยม เอาไปเรียกแบบฟังก์ชันไม่ได้" นั่นคือหลักฐานว่า @property ทำอะไร: พอแปะเข้าไป m.annual_vol ก็ไม่ใช่ฟังก์ชันอีกต่อไป มันคือผลลัพธ์ที่คำนวณเสร็จแล้ว

เก็บวิธีอ่านนี้ไว้ใช้ต่อ — เวลาเจอ 'X' object is not callable ให้แปลว่า "คุณใส่ () ต่อท้ายของที่ไม่ใช่ฟังก์ชัน" · สาเหตุที่พบบ่อยที่สุดสองข้อคือ เผลอใส่ () ให้ property กับ ตั้งชื่อตัวแปรทับชื่อฟังก์ชันไปแล้ว (เช่น sum = 0 แล้วเรียก sum(...) ที่หลัง)

📌อ่านว่า: คลาสข้างล่างมี 3 @property · วิธีอ่านที่ตรงที่สุดคือ "ช่องข้อมูลที่คิดสดใหม่ทุกครั้งที่มีคนถาม" — เก็บไว้เฉพาะของที่บอกกันมา (entry_price, size, exit_price) ส่วนของที่คำนวณต่อได้ (pnl, return_pct,

`is_open`) ไม่เก็บ แต่คิดตอนถูกเรียก · ข้อดีที่ได้ฟรีคือไม่มีวันไม่ตรงกัน — ถ้าเก็บ `self.pnl` ไว้เป็นตัวแปรแล้วมีคนแก้ไข `exit_price` ที่หลัง ตัวเลขเก่าจะค้างอยู่โดยไม่มีใครรู้

ตัวอย่าง: Position class พร้อม computed properties

```
class Position: """แทนตำแหน่งการเทรดเดี่ยว""" def __init__(self, symbol, side, size, entry_price, fee_rate: float = 0.001): self.symbol = symbol self.side = side # "long" หรือ "short" self.size = size # จำนวน unit self.entry_price = entry_price self.exit_price = None self.fee_rate = fee_rate @property def is_open(self): return self.exit_price is None @property def pnl(self): if self.exit_price is None: raise ValueError("Position ยังไม่ปิด") raw = (self.exit_price - self.entry_price) * self.size return raw if self.side == "long" else -raw @property def return_pct(self): return self.pnl / (self.entry_price * self.size) def close(self, exit_price): self.exit_price = exit_price return self def __repr__(self): status = "OPEN" if self.is_open else f"CLOSED PnL={self.pnl:+,.0f}" return f"Position({self.side} {self.size} {self.symbol} @ {self.entry_price:,.0f} [{status}])" # ทดสอบ pos = Position("BTCUSDT", "long", 0.5, 63000) print(pos) pos.close(65500) print(pos) print(f"Return: {pos.return_pct:.2%}")
```

► OUTPUT

```
Position(long 0.5 BTCUSDT @ 63,000 [OPEN])
Position(long 0.5 BTCUSDT @ 63,000 [CLOSED PnL=+1,250])
Return: 3.97%
```

🤖 4 ทำที่คลาสนี้ใช้ และจะเจอซ้ำในทุกคลาสหลังจากนี้

```
@property
def pnl(self):
    if self.exit_price is None:
        raise ValueError("Position ยังไม่ปิด") ① property ที่ปฏิเสธได้
    raw = (self.exit_price - self.entry_price) * self.size
    return raw if self.side == "long" else -raw ② พลิกเครื่องหมายให้ short

def close(self, exit_price):
    self.exit_price = exit_price
    return self ③ คืนตัวเอง = ต่อจุดได้

def __repr__(self): ④ หน้าตาตอนถูก print
    ...
```


1. **property ที่ raise ได้** — ถ้า `pnl` ของ `position` ที่ยังไม่ปิด แล้วโปรแกรมหยุดทันทีพร้อมบอกเหตุผล · ดีกว่า `คืน 0` หรือ `None` มาก เพราะ `0` จะไหลเข้าไปรวมใน `P&L` ของพอร์ตเจียบ ๆ แล้วคุณก็จะเห็นกำไรรวมที่ผิดโดยไม่มีอะไรเตือน
2. `raw if self.side == "long" else -raw` — **conditional expression** อ่านเรียงตามทีเขียน: "เอา `raw` ถ้าเป็น `long` ไม่งั้นเอา `-raw`" · เหตุผลทางการเงินคือ `short` กำไรตอนราคาลง ซึ่ง `exit - entry` จะติดลบพอดี จึงพลิกเครื่องหมายที่เดียวจบ แทนที่จะเขียนสูตรแยกสองชุด
3. `return self` ทำ `close()` — ทำให้เขียนต่อกันเป็นสายได้ เช่น `Position(...).close(65500).pnl` · เรียกว่า **method chaining** · ต้นทุนคือ `close()` ทำสองหน้าที่ (แก้ของ + คืนของ) ซึ่งบางทีก็ถือว่าไม่ควรทำ — รู้ไว้ว่ามีสองสำนัก
4. `__repr__` — กำหนดว่า `print(pos)` จะแสดงอะไร · ถ้าไม่เขียนจะได้ `<__main__.Position object at 0x7f3c...>` ซึ่งไร้ประโยชน์ตอนไล่ปัญหา · กฎที่ใช้ได้เสมอ: `__repr__` ควรใส่ค่าที่ทำให้คุณระบุตัวมันได้ ไม่ใช่คำบรรยายสวย ๆ

⚠ `fee_rate` ในคลาสนี้ถูกเก็บไว้เฉย ๆ — ไม่มีใครใช้

`__init__` รับ `fee_rate` เข้ามาและเก็บใส่ `self.fee_rate` แต่ทั้ง `pnl` และ `return_pct` ไม่ได้แตะมันเลย · ตัวเลข `PnL=+1,250` กับ `Return: 3.97%` ที่เห็นจึงเป็นผลแบบไม่มีค่าธรรมเนียม ทั้งที่หน้าตาของคลาสบอกไว้ชัดให้แล้ว (แบบฝึกหัดข้างล่างคือการเติมส่วนที่ขาดนี้)

ยกมาให้ดูเพราะนี่คือรูปแบบของบั๊กที่หาเจอยากที่สุดใน OOP — พารามิเตอร์ที่ถูกรับและถูกเก็บ แต่ไม่เคยถูกใช้ · โค้ดรันผ่าน ไม่มี warning ไม่มี error และชื่อตัวแปรทำหน้าที่โกหกแทน · คนอ่านเห็น `fee_rate` ในลายเซ็นก็เชื่อไปแล้วว่าหักให้

✖ **วิธีจับ:** ทุกครั้งที่เขียน `self.x = x` ใน `__init__` ให้ค้นหา `self.x` ในคลาสทันที · ถ้าเจอทีเดียวนั่นคือบรรทัดที่เพิ่งเขียน แปลว่ามันยังไม่ได้ทำงาน — ไม่ก็ยังไม่เสร็จ ไม่ก็ควรลบทิ้ง

● [รอบสอง] ราคาที่ต้องจ่ายของ property ที่ raise ได้

`pnl` โยน `ValueError` ตอน `position` ยังเปิดอยู่ ซึ่งถูกต้องในแง่ตรรกะ แต่มีผลข้างเคียงที่ควรรู้ล่วงหน้า: การ "แค่ดูค่า" กลายเป็นการกระทำที่ล้มเหลวได้

- ใน debugger หรือ notebook ที่ไล่แสดง attribute ทุกตัวของ object ให้อัตโนมัติ การมองก็ทำให้เกิด exception ได้
- เครื่องมือที่แปลง object เป็น dict/JSON แบบวนทุก property จะพังทันทีที่เจอ `position` ที่ยังไม่ปิดอยู่
- `__repr__` ในคลาสนี้รอดมาได้เพราะเช็ค `is_open` ก่อนแตะ `pnl` — ถ้าเขียน `__repr__` ให้พิมพ์ `pnl` ตรง ๆ การ `print` object ที่ยังไม่ปิดจะระเบิด และการ `print` ที่ระเบิดคือฝืนร้ายตอนไล่บั๊ก เพราะเครื่องมือที่ใช้หาปัญหากลายเป็นตัวสร้างปัญหาเสียเอง

ทางเลือกที่ทีมใหญ่มักใช้: ให้ property คืน `None` เมื่อยังไม่มีคำตอบ แล้วบังคับให้คนเรียกตัดสินใจอย่างชัดเจนว่าจะทำอะไรกับ `None` · หรือแยกเป็นสองชื่อ — `pnl` (raise) กับ `pnl_or_none` · ไม่มีคำตอบถูกข้อเดียว แต่ต้องเลือกให้เหมือนกันทั้งโปรเจกต์ ไม่งั้นคนใช้จะเดาไม่ถูกว่าต้องกัน exception ตรงไหนบ้าง

► แบบฝึกหัด: เพิ่ม `fee calculation` ให้ `Position`

อย่าเพิ่งข้ามแบบฝึกหัดข้างบน — โค้ดที่เหลือทั้ง Part ใช้เวอร์ชันนั้น

เฉลยข้างบนไม่ใช่ของเสริม · มันคือ `Position` ตัวที่หนังสือใช้ต่อไปจนจบ Part III เพราะเป็นตัวที่มี `net_pnl` (กำไรหลังหักค่าธรรมเนียมทั้งขาเข้าและขาออก) · ตั้งแต่บทที่ 19 เป็นต้นไปจะเห็น `position.net_pnl` โผล่ตลอด — ทั้งใน `BaseStrategy.log_trade` และ `Portfolio.metrics`

ถ้ารันตามในไฟล์เดียวแล้วข้ามบล็อกนั้นไป จะเจอ:

```
AttributeError: 'Position' object has no attribute 'net_pnl'
```

ซึ่งอ่านยากมากถ้าไม่รู้ที่มา เพราะ error จะโผล่ที่บทที่ 19 หรือ 21 — ห่างจากต้นเหตุจริงหลายสิบบรรทัด · กางเฉลยแล้วรันบล็อกนั้นก่อน แล้วค่อยอ่านต่อ

สรุปสัญญาของสองเวอร์ชันไว้เทียบ:

	<code>Position</code> ในเนื้อหาหลัก (18.4)	<code>Position</code> ในเฉลย (ใช้ต่อจากนี้)
เก็บ <code>fee_rate</code>	เก็บ แต่ไม่ได้ใช้	เก็บ และใช้จริง
<code>pnl</code>	กำไรก่อนหักค่าธรรมเนียม	—
<code>gross_pnl / net_pnl</code>	ไม่มี	มีทั้งคู่ · <code>net</code> หัก fee สองขา

บทที่ 19: Inheritance & Abstract Base Class

แก่นของบทนี้

Inheritance ให้ class ลูกรับ attribute และ method จาก class แม่ — ป้องกัน code ซ้ำ Abstract Base Class (ABC) กำหนด "สัญญา" ว่า class ลูกต้อง implement method ใดบ้าง เหมือน interface ใน Java/C++

19.1 Inheritance พื้นฐาน

```
class BaseStrategy: """Base class สำหรับทุก trading strategy""" def __init__(self, name, capital=100_000): self.name = name self.capital = capital self.trades = [] # history ของทุก trade self.equity = [capital] # equity curve def log_trade(self, position: Position): """บันทึก trade ที่ปิดแล้ว""" if position.is_open: raise ValueError("position นี้ยังไม่ถูกปิด - ต้อง close() ก่อน") self.trades.append(position) self.capital += position.net_pnl self.equity.append(self.capital) def summary(self): if not self.trades: print(f"{self.name}: ยังไม่มี trade") return pnls = [t.net_pnl for t in self.trades] winners = [p for p in pnls if p > 0] losers = [p for p in pnls if p <= 0] print(f"=== {self.name} ===") print(f"Trades: {len(self.trades)}") print(f"Win Rate: {len(winners)/len(pnls):.1%}") print(f"Avg Win: {sum(winners)/len(winners) if winners else 0:+, .0f}") print(f"Avg Loss: {sum(losers)/len(losers) if losers else 0:+, .0f}") print(f"Total PnL: {sum(pnls):+, .0f}") print(f"Final Capital: {self.capital:+, .0f}") def __repr__(self): return f"{self.__class__.__name__} ({self.name}, capital={self.capital:+, .0f})" class PairTradingStrategy(BaseStrategy): """Pair trading ต่อยอดจาก BaseStrategy""" def __init__(self, name, pair_config: PairConfig, capital=100_000): super().__init__(name, capital) # เรียก __init__ ของแม่ self.config = pair_config self.signal_engine = None def fit(self, price_a, price_b): """เทรน signal engine""" # PairTradingSignal ถูก import สมมติจาก Part II - ในโปรเจกต์จริงให้วางในไฟล์เดียวกัน self.signal_engine = PairTradingSignal( window = self.config.window, entry_z = self.config.entry_z, exit_z = self.config.exit_z ).fit(price_a, price_b) return self def get_signal(self): if self.signal_engine is None: raise RuntimeError("ต้อง fit() ก่อน") return self.signal_engine.signal().iloc[-1] # ทดสอบ inheritance cfg = PairConfig("BTC-Bybit", "BTC-Binance") strategy = PairTradingStrategy("BTC Pair v1", cfg, capital=500_000) print(strategy) print(f"เป็น BaseStrategy ไหม? {isinstance(strategy, BaseStrategy)}")
```

► OUTPUT

```
PairTradingStrategy(BTC Pair v1, capital=500,000)
เป็น BaseStrategy ไหม? True
```

🐞 ฝึกอ่าน: `super().__init__(name, capital)`

```
class PairTradingStrategy(BaseStrategy):
    def __init__(self, name, pair_config, capital=100_000):
        super().__init__(name, capital) # ← บรรทัดนี้
```

อ่านจากในออกนอก:

1. `super()` — "คลาสแม่ของฉัน" = `BaseStrategy` (ที่อยู่ใน parentheses ของ class definition)
2. `super().__init__` — "เรียก `__init__` ของ คลาสแม่"
3. `super().__init__(name, capital)` — ส่ง `name` กับ `capital` ให้แม่ไปตั้งค่า `self.name`, `self.capital`, `self.trades`, `self.equity` เหมือนที่แม่ทำ

ถ้าไม่เรียก `super().__init__()` → `self.name` และ `self.capital` จะไม่ถูกสร้าง → เรียกทีหลังจะ `AttributeError` — ลูกต้องเรียกแม่ให้ทำงานของแม่ก่อน แล้วค่อยทำงานของตัวเอง

19.2 Abstract Base Class — บังคับให้ implement

```
from abc import ABC, abstractmethod
class Strategy(ABC):
    """Abstract base class – blueprint สำหรับทุก strategy"""
    def __init__(self, name, capital=100_000):
        self.name = name
        self.capital = capital
        self.trades = []
    @abstractmethod
    def fit(self, data):
        """ต้อง implement – train model"""
    @abstractmethod
    def signal(self, data) -> str:
        """ต้อง implement – คืน 'LONG', 'SHORT', 'EXIT', 'HOLD'"""
    @abstractmethod
    def position_size(self, signal: str) -> float:
        """ต้อง implement – คืน size เป็น fraction ของ capital"""
    def summary(self): # concrete method – ไม่ต้อง override
        pnls = [t.net_pnl for t in self.trades]
        total = sum(pnls)
        if pnls else 0
        print(f"{self.name}: {len(pnls)} trades, PnL={total:+.0f}")
    def __repr__(self): # concrete
        """เหมือนกัน – ลูกทุกตัวได้ไปใช้ฟรี"""
        return (f"{type(self).__name__}" f"({self.name}, capital={self.capital:+.0f})")
    # ถ้าลืม implement abstractmethod จะ error ทันที
class BadStrategy(Strategy):
    pass
# ไม่ implement fit, signal, position_size
s = BadStrategy("test") # TypeError: Can't instantiate abstract class
```

concrete method คือเมธอดที่ไม่มี `@abstractmethod` — class แม่เขียน implementation ไว้ครบแล้ว class ลูกใช้ได้เลยโดยไม่ต้องเขียนซ้ำ (ตรงข้ามกับ abstract method ที่ลูกต้องเขียนเอง)

📌 **อ่านว่า:** ดู `__repr__` ในคลาสแม่ให้ดี — มันใช้ `type(self).__name__` ไม่ใช่เขียน "Strategy" ตรงๆ. `self` ตอนรันคือลูก ไม่ใช่แม่ ดังนั้น `PairStrategy` จะพิมพ์ว่า `PairStrategy(...)` และ `MomentumStrategy` จะพิมพ์ว่า `MomentumStrategy(...)` โดยที่เขียนโค้ดแค่ครั้งเดียวในแม่ — นี่คือประโยชน์ของ inheritance ที่เห็นชัดที่สุดในไฟล์นี้

🐞 ฝึกอ่าน: `@abstractmethod` และ `ABC`

```

from abc import ABC, abstractmethod

class Strategy(ABC):          # ABC = Abstract Base Class
    @abstractmethod
    def signal(self, data) -> str:
        pass

```

อ่านทีละส่วน:

1. ABC ใน (ABC) — บอก Python ว่า "class นี้เป็น abstract — ห้ามสร้าง object โดยตรง"
2. @abstractmethod — decorator ที่ "ติดป้าย" ว่า method นี้ **ต้องถูก override โดยลูก** — ถ้าลูกไม่ทำ สร้าง object ไม่ได้เลย
3. pass ใน method body — บอกว่า "แม้ไม่ได้ implement จริง ๆ แต่กำหนดชื่อ/signature ไว้"

เปรียบ ABC กับ interface ของหมอ: "ทุกคนที่จบแพทย์ ต้องมี license — ถ้าไม่มีจะทำงานเป็นหมอไม่ได้" — ABC คือการบังคับแบบเดียวกัน: ทุก strategy ต้อง implement ทั้ง `fit()`, `signal()` และ `position_size()`

ประโยชน์ของ ABC ใน trading system

- บังคับว่าทุก strategy ต้องมี `fit()`, `signal()`, `position_size()`
- Backtester ทำงานกับ Strategy ประเภทไหนก็ได้ — ไม่ต้องรู้ว่าเป็น pair, momentum, หรือ vol
- เพิ่ม strategy ใหม่ → implement 3 methods → ทำงานกับ backtester ทันที

19.3 Encapsulation — ซ่อน implementation detail

```

class RiskManager: """จัดการ risk — ซ่อน implementation ภายใน"""
    def __init__(self, max_position_pct=0.20, max_drawdown_pct=0.10):
        self._max_pos = max_position_pct # _ = "private by convention"
        self._max_dd = max_drawdown_pct
        self.__peak_equity = None # __ = name-mangled (private จริงๆ)
        def check(self, signal, capital, current_equity):
            """คืน True ถ้า trade ผ่าน risk check"""
            if signal == "HOLD": return False # อัปเดต peak
            if self.__peak_equity is None or current_equity > self.__peak_equity:
                self.__peak_equity = current_equity
            # Drawdown check
            drawdown = (current_equity - self.__peak_equity) / self.__peak_equity
            if drawdown < -self._max_dd:
                print(f"RISK BLOCK: Drawdown {drawdown:.1%} เกิน limit {-self._max_dd:.1%}")
                return False
            return True
        def size(self, signal, capital, volatility):
            """คำนวณ position size ที่ปลอดภัย"""
            if volatility <= 0: return 0 # Vol-adjusted sizing: target 1% daily vol contribution
            target_vol = 0.01
            raw_size = target_vol / volatility
            return min(raw_size, self._max_pos) # ไม่เกิน max position
        @property
        def max_position(self):
            return self._max_pos # max_position อ่านได้ แต่เปลี่ยนตรงๆ ไม่ควร
        rm = RiskManager(max_position_pct=0.15, max_drawdown_pct=0.10)
        print(f"Max position: {rm.max_position:.0%}")
        print(f"Approved: {rm.check('LONG', 500000, 490000)}")

```

📖 **อ่านว่า:** คลาสข้างล่างยาว 55 บรรทัด แต่โคตรงมีแค่ 2 ช่วงเวลา — จับให้ได้แล้วอ่านไพล่รวดเดียว: `fit()` คือ "เรียนจากอดีต ทำครั้งเดียว" · `signal()` คือ "ตัดสินใจกับแท่งล่าสุด ทำทุกแท่ง" · ทุกบรรทัดใน 55 บรรทัดนั้นอยู่ในช่วงใดช่วงหนึ่งเสมอ · และเส้นแบ่งนี้เองคือสิ่งที่กันการมองอนาคต — ถ้าเผลอเอาการคำนวณที่ต้องใช้ข้อมูลทั้งชุดไปไว้ใน `signal()` เมื่อไร กลยุทธ์จะเริ่มโกงทันทีโดยไม่มี error ใด ๆ

ตัวอย่าง: PairStrategy ที่ implement ABC ครบ

```
# ต้อง import เพิ่ม: import numpy as np · import pandas as pd # import
statsmodels.api as sm
class PairStrategy(Strategy): """Concrete pair trading
strategy – implement ครบ 3 abstract methods"""
    def __init__(self, name, symbol_a, symbol_b, window=60, entry_z=2.0, capital=100_000):
        super().__init__(name, capital)
        self.symbol_a = symbol_a
        self.symbol_b = symbol_b
        self.window = window
        self.entry_z = entry_z
        self._alpha = None
        self._beta = None
        self._mu = None
        self._sd = None
    def fit(self, data: pd.DataFrame): """เรียนความสัมพันธ์จากข้อมูลอดีต – ทำครั้งเดียว (หรือนาน ๆ ครั้ง) เก็บ 4 ค่า:
        alpha, beta (ความสัมพันธ์) + mu, sd (ระดับปกติของ spread) """
        log_a = np.log(data[self.symbol_a])
        log_b = np.log(data[self.symbol_b])
        X = sm.add_constant(pd.DataFrame({"log_b": log_b}))
        res = sm.OLS(log_a, X).fit()
        self._alpha = float(res.params["const"])
        self._beta = float(res.params["log_b"])
        resid = log_a - (self._alpha + self._beta * log_b)
        self._mu = float(resid.mean())
        self._sd = float(resid.std())
        if self._sd == 0 or np.isnan(self._sd):
            raise ValueError("spread ไม่มีความผันผวน – ข้อมูล train ผิดปกติ")
        return self
    def signal(self, data=None) -> str: """ให้คะแนนแท่งล่าสุด – เรียกได้ทุก timestep โดยไม่ต้อง fit ใหม่"""
        if self._beta is None:
            raise RuntimeError("ต้องเรียก fit() ก่อน signal()")
        if data is None or len(data) == 0:
            return "HOLD"
        # ใช้เฉพาะราคา "แท่งล่าสุด" ที่ส่งเข้ามา – ไม่แตะข้อมูลอนาคต
        log_a = np.log(data[self.symbol_a].iloc[-1])
        log_b = np.log(data[self.symbol_b].iloc[-1])
        spread = log_a - (self._alpha + self._beta * log_b)
        z = (spread - self._mu) / self._sd
        if z > self.entry_z:
            return "SHORT"
        if z < -self.entry_z:
            return "LONG"
        if abs(z) < 0.5:
            return "EXIT"
        return "HOLD"
    def position_size(self, signal: str) -> float:
        if signal == "HOLD":
            return 0.0
        if signal == "EXIT":
            return 0.0
        return 0.10 # 10% ของ capital (จะ optimize ใน Part IV)
# ตรวจสอบว่า implement ครบ
strat = PairStrategy("BTC Pair", "Bybit", "Binance")
print(isinstance(strat, Strategy)) # True
print(strat)
```

🤖 แพตเทิร์น "วัตถุสองเฟส" — ยังไม่พร้อม แล้วค่อยพร้อม

```
def __init__(self, ...):
    super().__init__(name, capital)
    self._alpha = None
    self._beta = None
    self._mu = None
```

① ให้คลาสแม่ตั้งของของมันก่อน
② จอ่งที่ไว้ — ยังไม่มีค่า

```

self._sd = None

def fit(self, data):
    ...
    return self

def signal(self, data=None):
    if self._beta is None:
        raise RuntimeError("ต้องเรียก fit() ก่อน signal()")

```

③ เฟสเรียน: เดิมค่าทั้ง 4

④ ยาม: กันใช้ก่อนพร้อม

1. `super().__init__(name, capital)` — เรียก `__init__` ของ **คลาสแม่** ให้ตั้งค่าส่วนที่เป็นของกลาง (ชื่อ · เงินทุน) ก่อน แล้วเราค่อยตั้งของเฉพาะตัว · **ลิมบรทัดนี้ = ของที่คลาสแม่ต้องตั้งจะไม่มีเลย** แล้วจะไปพึ่งที่หลัง เป็น `AttributeError` ที่ดูไม่ออกว่าเกี่ยวอะไรกับ `__init__`
2. ตั้งเป็น **None** ไว้ก่อน ทั้งที่ยังไม่มีค่า — ไม่ใช่ความสับสนเปลือง แต่เป็นการประกาศว่าวัตถุนี้อาจจะมีอะไรบ้าง · คนอ่าน `__init__` อย่างเดียวก็เห็นคร่าวๆ คลาสนี้จะทำอะไรไว้ · ถ้าไม่ประกาศแล้วไปสร้างเอาใน `fit()` คนอ่านต้องไล่ทั้งคลาสถึงจะรู้
3. `_` นำหน้าชื่อ (`_alpha`, `_beta`) — เป็นข้อตกลงระหว่างคน ว่า "ของภายใน อย่าแตะจากข้างนอก" · Python ไม่ได้บังคับอะไรเลย แต่ได้จริง · แต่พอเห็น `_` ก็รู้ทันทีว่าถ้าไปแก้เอง อาจพังโดยที่คนเขียนไม่ได้รับประกันอะไรให้
4. ยามที่บรรทัดแรกของ `signal()` — `if self._beta is None: raise` · นี่คือการทำให้แพดเทิร์นสองเฟส *ปลอดภัย* · ถ้าไม่มียามตัวนี้การเรียก `signal()` ก่อน `fit()` จะไม่ error แต่จะไปคำนวณด้วย `None` แล้วได้ `TypeError` ลึก ๆ กลางสูตร ซึ่งอ่านไม่ออกว่าต้นเหตุคือลิม `fit()`

`fit()` จบด้วย `return self` เหมือน `Position.close()` — เขียน `PairStrategy(...).fit(train).signal(latest)` ต่อกันได้แบบบรรทัดเดียว

● [รอบสอง] `mu` กับ `sd` ถูกแซ่แข็งไว้ตั้งแต่วันที่ `fit` — และนั่นคือจุดที่กลยุทธ์คู่ตายจริง

ดูให้ดีๆ `signal()` เอา spread ของแท่งล่าสุดไปเทียบกับ `self._mu` และ `self._sd` ที่คำนวณไว้ตั้งแต่ตอน `fit()` · トラバิดที่ยังไม่เรียก `fit()` ใหม่ ตัวเลขสองตัวนี้จะไม่ขยับอีกเลย

ผลที่ตามมาเมื่อความสัมพันธ์ของคู่เหรียญเปลี่ยนไปจริง ๆ (ซึ่งเกิดเสมอ — เปลี่ยนค่าธรรมเนียม เปลี่ยนสภาพคล่อง มีคนถอนสภาพคล่องออก):

- spread จริงเลื่อนไปอยู่ระดับใหม่ แต่ `mu` ยังชี้ไปที่ระดับเก่า
- `z` จึงค้างอยู่ฝั่งเดียวถาวร → กลยุทธ์เปิดไม้เดิมซ้ำ ๆ แล้วไม่เคยได้สัญญาณ EXIT เพราะ `abs(z) < 0.5` ไม่มีวันเป็นจริงอีก
- อาการที่เห็นจากภายนอกคือ "กลยุทธ์เคยดีแล้วจู่ ๆ ก็ค้างสถานะยาว" — ไม่มี error ไม่มี log ผิดปกติ

ทางแก้มี 2 แนว และเลือกได้ตามว่าคุณเชื่ออะไร:

1. **fit ใหม่เป็นระยะ** (เช่น ทุกสัปดาห์ ด้วยข้อมูลย้อนหลังหน้าต่างคงที่) — ง่ายสุด และยังคงได้ว่า "เรียนจากอดีตเท่านั้น" อยู่

2. ใช้ค่าที่ขยับตามเวลา — `rolling mean/std` หรือ `RollingOLS` สำหรับ `beta` · แม่นกว่าแต่ต้องระวัง
มองอนาคตมากเกินไป เพราะการคำนวณย้ายเข้าไปอยู่ในเส้นทางของ `signal()` แล้ว

และไม่ว่าจะเลือกทางไหน สิ่งที่ต้องมีคู่กันเสมอคือ ตัวเฝ้าดูว่า `spread` ยังอยู่ในระดับที่เคยเรียนมาไหม — ถ้า
หลุดกรอบไปนานผิดปกติ ให้หยุดเทรดคู่ นั้น ไม่ใช่ปล่อยให้มันถล่นต่อไปเรื่อย ๆ

► แบบฝึกหัด: สร้าง `MomentumStrategy` ที่ implement ABC

บทที่ 20: Design Patterns

แก่นของบทนี้

Design Pattern คือ "สูตรสำเร็จ" ที่นักพัฒนาทั่วโลกใช้แก้ปัญหาเดิม ๆ มาหลายสิบปี — ไม่ต้องคิดใหม่ตั้งแต่ต้น ใน Quant system ใช้บ่อย 3 patterns: **Strategy Pattern** (สลับ algorithm ได้โดยไม่แก้ backtester), **Observer Pattern** (แจ้งเตือนเมื่อมี event เช่น signal เกิด), **Factory Pattern** (สร้าง object จาก config โดยไม่ hard-code ชื่อ class)

20.1 Strategy Pattern — สลับ Algorithm

Backtester ทำงานกับ *ทุก* strategy โดยไม่รู้ว่าเป็นประเภทไหน — ขอแค่ implement `Strategy ABC`

```
class Backtester: """ Generic backtester — ทำงานกับ Strategy ใดก็ได้ Strategy Pattern:
algorithm (strategy) แยกจาก runner (backtester) """
    def __init__(self, strategy: Strategy, risk_manager: RiskManager = None):
        self.strategy = strategy
        self.rm = risk_manager or RiskManager()
        self.results = []
        def run(self, data: pd.DataFrame, price_col: str) -> pd.DataFrame:
            """ รัน backtest บน data data: DataFrame ที่มีราคา price_col: ชื่อ column ของราคาที่ใช้ execute trade """
            prices = data[price_col].values
            capital = self.strategy.capital
            equity = [capital]
            pos = None # position ปัจจุบัน
            self.strategy.fit(data.iloc[:60]) # train บน 60 วันแรก
            for i in range(60, len(data)):
                window_data = data.iloc[max(0, i-60):i]
                sig = self.strategy.signal(window_data)
                price = prices[i]
                # Exit logic
                if pos and (sig == "EXIT" or (sig == "LONG" and pos.side == "short") or (sig == "SHORT" and pos.side == "long")):
                    pos.close(price)
                    capital += pos.net_pnl
                    self.strategy.trades.append(pos)
                    pos = None
                # Entry logic
                if pos is None and sig in ("LONG", "SHORT"):
                    if self.rm.check(sig, capital, capital):
                        frac = self.strategy.position_size(sig)
                        size = (capital * frac) / price
                        side = "long" if sig == "LONG" else "short"
                        pos = Position(price_col, side, size, price)
                    equity.append(capital)
                data = data.copy()
                data["equity"] = equity[:len(data)]
            return data
        # ใช้งาน Strategy Pattern: # สลับระหว่าง pair กับ momentum โดยไม่แก้ Backtester เลย
        pair_bt = Backtester(PairStrategy("BTC Pair", "Bybit", "Binance"))
        mom_bt = Backtester(MomentumStrategy("BTC Mom", "Bybit"))
```

20.2 Observer Pattern — Event System

Observer ให้ object "สมัครรับ" event จาก object อื่น — เมื่อ event เกิด ทุก observer ถูกแจ้ง

```
from typing import Callable, List
class EventBus: """ Simple event bus — Observer Pattern
ใช้สำหรับ: signal เกิด → แจ้ง logger, risk manager, order executor """
    def __init__(self):
        self._listeners: dict[str, List[Callable]] = {}
        def subscribe(self,
```

```

event: str, callback: Callable): """สมัครรับ event""" self._listeners.setdefault(event, []).append(callback) def publish(self, event: str, data=None): """ส่ง event ให้ทุก subscriber""" for cb in self._listeners.get(event, []): cb(data) # ตัวอย่าง: เมื่อ signal เกิด → logger, risk, executor ทำงาน
bus = EventBus() # Subscribers
def on_signal_logger(data): print(f"[LOG] Signal: {data['signal']} @ {data['price']:, .0f}")
def on_signal_risk(data): if data['signal'] != "HOLD": print(f"[RISK] Checking signal: {data['signal']}")
def on_signal_execute(data): if data['signal'] == "LONG": print(f"[EXEC] Placing LONG order @ {data['price']:, .0f}")
bus.subscribe("signal", on_signal_logger)
bus.subscribe("signal", on_signal_risk)
bus.subscribe("signal", on_signal_execute) # Publish event
bus.publish("signal", {"signal": "LONG", "price": 65000})

```

► OUTPUT

```

[LOG] Signal: LONG @ 65,000
[RISK] Checking signal: LONG
[EXEC] Placing LONG order @ 65,000

```

20.3 Factory Pattern — สร้าง Object จาก Config

```

class StrategyFactory: """ Factory Pattern: สร้าง strategy object จาก config dict ป้องกัน hard-code ชื่อ class ใน code หลัก """
    _registry = {}
    @classmethod
    def register(cls, name: str, strategy_class):
        cls._registry[name] = strategy_class
    @classmethod
    def create(cls, config: dict) -> Strategy:
        strat_type = config.get("type")
        if strat_type not in cls._registry:
            raise ValueError(f"Unknown strategy: {strat_type}. "
                             f"Available: {list(cls._registry.keys())}")
        klass = cls._registry[strat_type]
        return klass(**{k: v for k, v in config.items() if k != "type"})
# ลงทะเบียน strategies
StrategyFactory.register("pair", PairStrategy)
StrategyFactory.register("momentum", MomentumStrategy)
# สร้างจาก config (อาจมาจาก JSON file)
config = { "type": "pair", "name": "BTC Pair v2", "symbol_a": "Bybit", "symbol_b": "Binance", "window": 60, "entry_z": 2.0, "capital": 500_000 }
strat = StrategyFactory.create(config)
print(strat)
print(type(strat).__name__)

```

► OUTPUT

```

PairStrategy(BTC Pair v2, capital=500,000)
PairStrategy

```

เมื่อไหร่ควรใช้ Design Pattern แต่ละแบบ

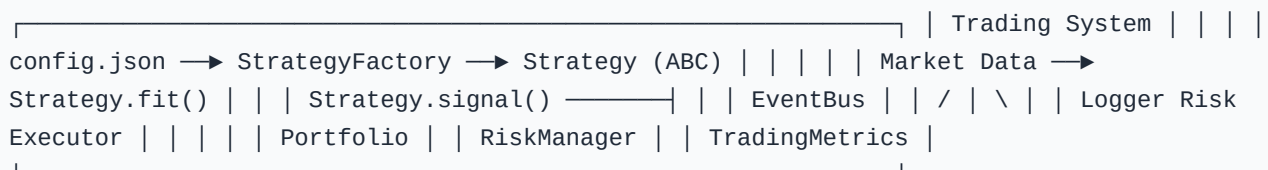
Pattern	ใช้เมื่อ	ตัวอย่างใน trading
Strategy	มีหลาย algorithm ที่สลับกันได้	Pair / Momentum / Volatility strategy
Observer	หลาย component ต้องรู้เมื่อ event เกิด	Signal → Log + Risk + Execute
Factory	สร้าง object จาก config โดยไม่ hard-code	Load strategy จาก JSON config

บทที่ 21: OOP Mini-Project — Trading System

แก่นของบทนี้

รวมทุกอย่างจาก Part III เป็น trading system ที่ใช้งานได้จริง: **Portfolio** (เก็บ positions หลายตัว), **RiskManager** (ควบคุม risk), **Backtester** (ทดสอบย้อนหลัง), **StrategyFactory** (สร้างจาก config) — ทั้งหมดเชื่อมกันด้วย Event Bus

21.1 Architecture Overview



21.2 Portfolio class

```
class Portfolio: """ จัดการ positions หลายตัวพร้อมกัน เก็บ state ทั้งหมดของระบบการเทรด """
    def __init__(self, initial_capital=1_000_000):
        self.initial_capital = initial_capital
        self.cash = initial_capital
        self.open_positions = {} # symbol -> Position
        self.closed_positions = []
        self._equity_history = [initial_capital]

    def open(self, symbol, side, size, price, fee_rate=0.001):
        """เปิด position ใหม่"""
        if symbol in self.open_positions:
            raise ValueError(f"มี position {symbol} อยู่แล้ว")
        # long = +1 . short = -1 . ตัวเดียวนั้นคุมทิศทางของเงินทั้งคลาส # long ควักเงินไปซื้อของ → cash ลด # short ขายของที่
        # ยืมมา ได้เงิน → cash เพิ่ม (แต่ติดหนี้เป็นของ)
        sign = 1 if side == "long" else -1
        notional = size * price
        fee = notional * fee_rate
        # ใช้ notional เป็นเกณฑ์ทั้งสองฝั่ง — short ก็ต้องมีเงินวาง
        # ประกัน if notional + fee > self.cash: raise ValueError( f"เงินไม่พอ: ต้องการ {notional +
        # fee:,.0f}, มี {self.cash:,.0f}" )
        self.cash -= sign * notional + fee
        self.open_positions[symbol] = Position(symbol, side, size, price, fee_rate)
        return self

    def close(self, symbol, price):
        """ปิด position"""
        if symbol not in self.open_positions:
            raise ValueError(f"ไม่มี position {symbol}")
        pos = self.open_positions.pop(symbol)
        pos.close(price) # ปิดคือทำกลับด้านของตอนเปิด — เครื่องหมายจึง
        # กลับ แต่ fee ยังจ่ายเสมอ
        sign = 1 if pos.side == "long" else -1
        notional = pos.size * price
        self.cash += sign * notional - notional * pos.fee_rate
        self.closed_positions.append(pos)
        self._equity_history.append(self.total_equity({symbol: price}))
        return pos

    def total_equity(self, current_prices: dict) -> float:
        """มูลค่ารวม = cash + มูลค่าของที่ถืออยู่ (short นับเป็นลบ)"""
        unrealized = sum( (1 if pos.side == "long" else -1) * pos.size *
                           current_prices.get(sym, pos.entry_price)
                           for sym, pos in self.open_positions.items())
```

```
) return self.cash + unrealized @property def metrics(self): pnls = [p.net_pnl for p
in self.closed_positions] if not pnls: return {"trades": 0} winners = [p for p in
pnls if p > 0] return { "trades": len(pnls), "win_rate": len(winners) / len(pnls),
"total_pnl": sum(pnls), "avg_win": sum(winners)/len(winners) if winners else 0,
"avg_loss": sum(p for p in pnls if p <= 0) / max(1, len(pnls)-len(winners)) } def
__repr__(self): return (f"Portfolio(cash={self.cash:,.0f}, " f"positions=
{list(self.open_positions.keys())}, " f"trades={len(self.closed_positions)}")"
```

sign ตัวเดียวคุมทิศทางของเงินทั้งคลาส

```
sign = 1 if side == "long" else -1

self.cash -= sign * notional + fee      # เปิด
self.cash += sign * notional - notional * fee_rate    # ปิด

unrealized = sum(sign(pos) * pos.size * ราคาปัจจุบัน ...) # ตีมูลค่า
```

1. **ทำไมต้องมี sign** — long กับ short เป็นสิ่งตรงข้ามกันในทุกมิติ: long ควักเงินไปซื้อของ (cash ลด แล้วได้ของมาถือ) · short ขายของที่ถือมา (cash เพิ่ม แต่ติดหนี้เป็นของที่ต้องซื้อคืน) · แทนที่จะเขียนสูตรแยกสองชุดแล้วเสี่ยงแก้มักรบใช้ตัวคูณ ± 1 ตัวเดียวคุมทั้งหมด
2. **fee ไม่คูณ sign** — สังเกตให้ดี · ทิศทางของเงินต้นกลับได้ แต่ค่าธรรมเนียมจ่ายออกเสมอทั้งขาเข้าและขาออก ทั้ง long และ short · ถ้าเฟลคูณ sign ไปด้วย จะกลายเป็น "short แล้วได้ค่าธรรมเนียมคืน" ซึ่งไม่มีในโลกจริง
3. **total_equity ต้องคูณ sign ด้วย** — เพราะของที่ถือ short มีมูลค่าติดลบต่อพอร์ต · ราคาขึ้น = หนี้เพิ่ม = equity ลด · ถ้าลืมจุดนี้ พอร์ตจะรายงานว่ากำไรตอนที่ยขาดทุนจริง
4. **คำนวณเงินใช้ notional ไม่ใช่กระแสเงินสด** — สำหรับ short กระแสเงินสดเป็นบวก (ได้เงินเข้า) ถ้าเช็คจากตัวนั้นจะเปิด short ขนาดเท่าไรก็ได้แม้เงินในบัญชีจะไม่พอปิดสถานะ · ในโลกจริงต้องวางเงินประกัน จึงเช็คจากขนาดสถานะ เป็นเกณฑ์ทั้งสองฝั่ง

วิธีตรวจว่าบัญชีถูกจริง คือดูว่า ตัวเลขจากสองที่ต้องตรงกัน: กำไรที่ **Position** รายงาน กับเงินสดที่ขยับจริงในพอร์ต · ถ้าไม่ตรงเมื่อไร แปลว่ามีเงินหายหรือออกอยู่ที่ไหนสักแห่ง

```
# ตรวจ short: เปิดที่ 100 แล้วราดาลงไป 90 → short ต้องกำไร pf =
Portfolio(initial_capital=1_000_000) pf.open("TEST", "short", size=100, price=100.0)
print(f"cash หลังเปิด short : {pf.cash:,.2f}") print(f"equity ตอนราคา 90 :
{pf.total_equity({'TEST': 90.0}):,.2f} ← ต้องกำไร") print(f"equity ตอนราคา 110 :
{pf.total_equity({'TEST': 110.0}):,.2f} ← ต้องขาดทุน") pos = pf.close("TEST", 90.0)
cash_moved = pf.cash - 1_000_000 print(f"\nPosition.net_pnl นอก :
{pos.net_pnl:+,.2f}") print(f"cash ขยับจริง : {cash_moved:+,.2f}") print("ตรงกัน ✓" if
abs(pos.net_pnl - cash_moved) < 1e-9 else "✗ ไม่ตรง - บัญชีรั่ว")
```

► OUTPUT

```
cash หลังเปิด short : 1,009,990.00
equity ตอนราคา 90 : 1,000,990.00 ← ต้องกำไร
equity ตอนราคา 110 : 998,990.00 ← ต้องขาดทุน

Position.net_pnl บวก : +981.00
cash ขยับจริง : +981.00
ตรงกัน ✓
```

เทคนิคที่ใช้ได้กับทุกระบบบัญชี — "ตรวจจากสองทาง"

บล็อกข้างบนไม่ได้ตรวจว่า "ตัวเลขเท่ากับ 981 ไหม" — ถ้าตรวจแบบนั้นต้องมานั่งแก้ทุกครั้งที่เปลี่ยนค่าธรรมเนียมหรือขนาดไม้. มันตรวจว่า **สองเส้นทางที่คำนวณคนละวิธีให้คำตอบตรงกันไหม**: ทางหนึ่งคิดจากราคาเข้า-ออกของ Position อีกทางคิดจากเงินสดที่ไหลเข้าออกพอร์ตจริง

เงื่อนไขแบบนี้เป็นจริงเสมอไม่ว่าตัวเลขจะเปลี่ยนไปแค่ไหน — เป็นการทดสอบชนิดเดียวกับที่บทที่ 28 เรียกว่า "ตรวจคุณสมบัติ ไม่ใช่ตรวจคำตอบ" . และในระบบเทรดจริงมันคือด่านสุดท้ายที่จับได้ว่า fee คิดขาดไปข้างหนึ่งหรือ short คิดกลับทิศ

🔗 ทุกครั้งที่เขียนโค้ดที่ขยับเงิน ให้หาตัวเลขเดียวกันที่คำนวณได้จากอีกทางเสมอ แล้วเทียบกัน . ถ้าหาไม่เจอ แปลว่าระบบนั้นยังไม่มีทางรู้เลยว่าตัวเองผิดหรือถูก

21.3 ประกอบทุก Component เป็น TradingSystem

TradingSystem ออกแบบตามแนวคิด **Facade** — เป็น "จุดติดต่อเดียว" ที่ซ่อนความซับซ้อนข้างในไว้ เหมือนรีโมททีวีที่กดปุ่มเดียวแต่ข้างในสั่งงานหลายวงจร ผู้ใช้ไม่ต้องรู้ว่า มี Strategy, Portfolio, RiskManager ทำงานประสานกันอย่างไร

```
class TradingSystem: """ Facade class — จัดการทุกอย่างผ่านจุดเดียว ใช้: system =
TradingSystem.from_config(config) """ def __init__(self, strategy: Strategy,
portfolio: Portfolio, risk_manager: RiskManager): self.strategy = strategy
self.portfolio = portfolio self.rm = risk_manager self.bus = EventBus()
self._setup_listeners() def _setup_listeners(self): self.bus.subscribe("signal",
self._on_signal) self.bus.subscribe("error", self._on_error) def _on_signal(self,
data): sig = data["signal"] price = data["price"] sym = data["symbol"] if sig ==
"LONG" and sym not in self.portfolio.open_positions: capital =
self.portfolio.total_equity({}) if self.rm.check(sig, capital, capital): frac =
self.strategy.position_size(sig) size = (capital * frac) / price
self.portfolio.open(sym, "long", size, price) print(f"[OPEN] LONG {sym} size=
{size:.4f} @ {price:,.0f}") elif sig in ("EXIT", "SHORT") and sym in
self.portfolio.open_positions: pos = self.portfolio.close(sym, price) print(f"[CLOSE]
{sym} PnL={pos.net_pnl:+,.0f}") def _on_error(self, data): print(f"[ERROR] {data}")
@classmethod def from_config(cls, config: dict): """สร้างระบบทั้งหมดจาก config dict"""
strategy = StrategyFactory.create(config["strategy"]) portfolio =
Portfolio(config.get("capital", 100_000)) risk_mgr = RiskManager(
```

```

max_position_pct=config.get("max_position", 0.20),
max_drawdown_pct=config.get("max_drawdown", 0.10) ) return cls(strategy, portfolio,
risk_mgr) def step(self, symbol, price, data): """ประมวลผล 1 timestep""" sig =
self.strategy.signal(data) self.bus.publish("signal", {"signal": sig, "price": price,
"symbol": symbol}) def report(self): m = self.portfolio.metrics print(f"\n=== Final
Report: {self.strategy.name} ===") for k, v in m.items(): if isinstance(v, float):
print(f" {k:12s}: {v:.2%}" if "rate" in k else f" {k:12s}: {v:+.0f}") else: print(f"
{k:12s}: {v}")

```

21.4 ทดลองใช้ระบบทั้งหมด

```

# Config ทั้งระบบในที่เดียว SYSTEM_CONFIG = { "capital": 1_000_000, "max_position": 0.15,
"max_drawdown": 0.10, "strategy": { "type": "pair", "name": "BTC Pair Production",
"symbol_a": "Bybit", "symbol_b": "Binance", "window": 60, "entry_z": 2.0, "capital":
1_000_000 } } # สร้างระบบ system = TradingSystem.from_config(SYSTEM_CONFIG) # จำลองข้อมูล
และวิ่ง import numpy as np, pandas as pd import statsmodels.api as sm # PairStrategy ข้าง
ในใช้ OLS from statsmodels.regression.rolling import RollingOLS np.random.seed(0) n =
300 # train 60 วัน + live 240 วัน dates = pd.date_range("2024-01-01", periods=n) bybit
= 65000 + np.cumsum(np.random.normal(0, 80, n)) # spread ต้องดึงกลับเข้าค่ากลาง (OU) ไม่ใช่
noise ลอย ๆ # ถ้าเป็น noise กลยุทธ์จะไม่มีอะไรให้จับ = แทบไม่เกิดเทรดเลย kappa, sigma_ou = 0.15,
45.0 ou = np.zeros(n) for t in range(1, n): ou[t] = ou[t-1] + kappa * (0 - ou[t-1]) +
np.random.normal(0, sigma_ou) binance = bybit + ou # ติดกับ bybit + spread ที่ mean-
revert df = pd.DataFrame({"Bybit": bybit, "Binance": binance}, index=dates) # Train
ครั้งแรก system.strategy.fit(df.iloc[:60]) # Live simulation (240 steps) REFIT_EVERY =
20 # refit ทุก 20 วัน print(f"=== Simulating {n-60} days ===") for i in range(60, n):
window = df.iloc[max(0, i-60):i] # ความสัมพันธ์ระหว่างสองตลาดเปลี่ยนไปเรื่อย ๆ - ต้อง refit เป็น
รอบ if i > 60 and (i - 60) % REFIT_EVERY == 0: system.strategy.fit(window) price =
df["Bybit"].iloc[i] system.step("Bybit", price, window) system.report()

```

► OUTPUT

```

=== Simulating 240 days ===
[OPEN] LONG Bybit size=1.5391 @ 64,971
[CLOSE] Bybit PnL=-200
[OPEN] LONG Bybit size=1.5429 @ 64,800
[CLOSE] Bybit PnL=-337
...
[CLOSE] Bybit PnL=-3

=== Final Report: BTC Pair Production ===
trades      : 6
win_rate    : 0.00%
total_pnl   : -860
avg_win     : 0
avg_loss    : -143

```

🔴 **อ่านว่า:** สังเกตบรรทัด `if i > 60 and (i - 60) % REFIT_EVERY == 0:`

`system.strategy.fit(window)` — เราเรียก `fit()` ซ้ำกลางทางได้เลย เพราะออกแบบให้ `fit()` (เรียนความสัมพันธ์) แยกจาก `signal()` (ให้คะแนนแท่งล่าสุด) · ถ้าเขียนแบบเอา z-score ทั้งชุดไป cache ไว้ใน `fit()` การ refit กลางทางจะยุ่งกว่านี้มาก — นี่คือผลตอบแทนของการแยกหน้าที่ให้ชัด ที่ Part นี้พยายามสอน

⚠️ อ่านตัวเลข PnL ของ mini-project นี้อย่างไร — อย่าตีความว่ากลยุทธ์ดีหรือแย่

ดู log จะเห็นว่าทุกเทรดเป็น LONG Bybit — **ชาเดียว** · เพราะ `system.step("Bybit", price, window)` ส่งเข้าไปแค่ symbol เดียวกับราคาเดียว ดังนั้น Portfolio จึงเปิดสถานะใน Bybit เท่านั้น ไม่ได้ short Binance คู่กัน

ผลคือ กำไร/ขาดทุนที่เห็นมาจากทิศทางราคา Bybit ไม่ใช่จากการบีบตัวของ spread — signal มาจาก spread แต่การ execute เป็น directional bet · ตัวเลข `total_pnl` จึงเป็นสัญญาณรบกวนล้วน ๆ **ไม่ว่าจะออกมาบวกหรือลบก็ไม่ได้บอกอะไรเกี่ยวกับคุณภาพของกลยุทธ์**

🎯 **สิ่งที่ mini-project นี้พิสูจน์สำเร็จคือ "โครงสร้าง" ไม่ใช่ "ผลตอบแทน":** config → Factory สร้าง strategy ได้ · EventBus ส่งต่อ signal ถึง risk manager และ executor ครบวงจร · Portfolio บันทึกทุกเทรดและสรุป metrics ได้ · เพิ่ม strategy ใหม่ไม่ต้องแก้ระบบ

🔧 **โจทย์ต่อยอด (ตรงนี้คือก้าวถัดไปของจริง):** ทำให้ Portfolio รับสถานะ **สองขา** ได้ — `step()` ต้องรับราคาทั้งคู่, การเปิดสถานะต้อง long ขาหนึ่ง short อีกขาด้วยอัตราส่วน `beta` ที่ `fit()` คำนวณไว้ และ PnL ต้องรวมสองขา · ทำได้แล้วตัวเลขในรายงานจะเริ่มมีความหมาย

จบ Part III — สิ่งที่ได้จาก OOP

ก่อน OOP (Part II): functions กระจาย, เพิ่ม strategy ใหม่ = แก้หลายไฟล์

หลัง OOP (Part III): เพิ่ม strategy ใหม่ = สร้าง class ใหม่ที่ implement ABC → ทำงานกับ Backtester และ TradingSystem ทันที

- Strategy ABC บังคับ interface ✓
- Portfolio เก็บ state ครบ ✓
- EventBus แยก concerns ✓
- StrategyFactory + config สร้างระบบจาก JSON ✓
- TradingSystem ประกอบทุกอย่างผ่านจุดเดียว ✓

หมายเหตุเรื่องประสิทธิภาพ: Backtester ใน Part III ใช้ Python for-loop ซึ่งช้ากว่า vectorized ใน Part IV ประมาณ **50–100x** ใช้เพื่อเรียนรู้ logic ได้ แต่สำหรับ parameter search ให้ใช้ vectorized ใน Part IV เท่านั้น

ใน **Part IV** เราจะนำ system นี้ไป backtest อย่างจริงจัง: walk-forward, slippage, commission, overfitting

[← Part II: Math Tools](#)

[Part IV: Backtesting →](#)